Complete Real-Time Chat Application Master Handbook

Interview Preparation • Viva Guide • Architecture • Algorithms • Flowcharts • Code Concepts

Project Introduction

This handbook explains the complete Real-Time Chat Application built using React.js, Node.js, Express.js, MySQL, JWT, bcrypt and Socket.io.

Problem Statement

Traditional HTTP applications cannot provide instant bidirectional communication. This project solves that by implementing real-time messaging using Socket.io.

Technology Stack

Frontend: React.js. Backend: Node.js & Express.js. Database: MySQL. Authentication: JWT. Password Security: bcrypt. Real-Time Communication: Socket.io.

System Architecture

React UI communicates with REST APIs and Socket.io. Express handles APIs, Socket.io handles real-time events, and MySQL stores users and messages.

Database Design

Users(id, username, password). Messages(id, sender_id, receiver_id, message, created_at).

Registration Flow

User enters credentials -> bcrypt hashes password -> password stored in MySQL -> registration successful.

Login Flow

User enters credentials -> database lookup -> bcrypt.compare -> JWT generation -> token returned to frontend.

JWT Deep Explanation

JWT contains header, payload and signature. The payload stores user identity. Tokens are verified using jwt.verify().

Socket Authentication

The frontend sends JWT during socket handshake. Backend middleware verifies the token before connection is allowed.

Online Users Algorithm

Connected users are stored in onlineUsers object. Every connect/disconnect broadcasts an updated user list.

Private Messaging Algorithm

Sender emits privateMessage -> backend stores message -> backend emits receiveMessage -> receiver updates UI.

Typing Indicator Algorithm

Sender emits typing -> backend forwards typing event -> receiver shows 'is typing...' message.

Chat History Algorithm

User selects chat -> loadMessages event -> SQL query fetches messages -> frontend displays history.

React Hooks

useState manages state. useEffect manages side effects and socket listeners. useRef supports auto-scroll.

Socket.io Concepts

Events, emit, on, rooms, namespaces, middleware, connection lifecycle.

Files Explained

App.js: UI and state. Login.js: login. Register.js: registration. auth.js: APIs. server.js: sockets and backend. db.js: database connection.

Flowchart: Registration

Start -> Enter Username & Password -> Hash Password -> Store User -> Success -> End.

Flowchart: Login

Start -> Enter Credentials -> Validate -> Generate JWT -> Store Token -> Success.

Flowchart: Messaging

Sender -> Socket Event -> Backend -> Database -> Receiver Room -> Display Message.

Flowchart: Authentication

Login -> Generate Token -> Store Token -> Socket Handshake -> Verify Token -> Connect.

Security Features

Password hashing, JWT verification, protected socket connections, secure authentication.

Common Bugs and Fixes

Invalid token, socket authentication error, EADDRINUSE, receiverId null, missing listeners, duplicate socket connections.

Interview Questions and Answers

Why JWT? Stateless authentication. Why bcrypt? Secure password hashing. Why Socket.io? Real-time communication.

Project Explanation (2 Minutes)

A concise explanation covering architecture, authentication, messaging and database integration.

Project Explanation (5 Minutes)

A detailed explanation suitable for technical interviews and project presentations.

Resume Description

Professional resume-ready project description.

GitHub README Structure

Overview, setup, technologies, screenshots, architecture and usage.

Future Enhancements

Group chat, file sharing, read receipts, dark mode, profile pictures.

60 Technical Interview Questions

• Explain your project architecture.
• What is React?
• Why React Hooks?
• What is useState?
• What is useEffect?
• What is useRef?
• What is Node.js?
• What is Express.js?
• What is MySQL?
• Why MySQL?
• What is JWT?
• How is JWT generated?
• How is JWT verified?
• What happens when JWT expires?
• What is bcrypt?
• Why hash passwords?
• What is localStorage?
• What is Socket.io?
• How is Socket.io different from HTTP?
• What is WebSocket?
• What is a socket room?
• Why use rooms?
• How does private messaging work?
• How do online users work?
• How does typing indicator work?
• How does chat history work?
• What is middleware?
• What is socket middleware?
• What is CORS?
• What is REST API?
• What is authentication?
• What is authorization?
• What is hashing?
• What is SQL JOIN?
• Why JOIN users table?
• What is auto-scroll?

• Why use timestamps?
• How are messages stored?
• What is receiverId?
• What caused receiverId null bug?
• What is EADDRINUSE?
• How did you debug issues?
• How do you secure sockets?
• How does logout work?
• What is event-driven programming?
• What is emit?
• What is on()?
• What is a callback?
• What is async/await?
• What is state management?
• What is component lifecycle?
• How does React re-render?
• What are advantages of JWT?
• Why not store plaintext passwords?
• What are future enhancements?
• How would you scale this project?
• What are limitations?
• What was the biggest challenge?
• What did you learn?